

[Sign Out](#)[Account](#)[Introduction to Deep RL and DQN](#)[16 min read](#)

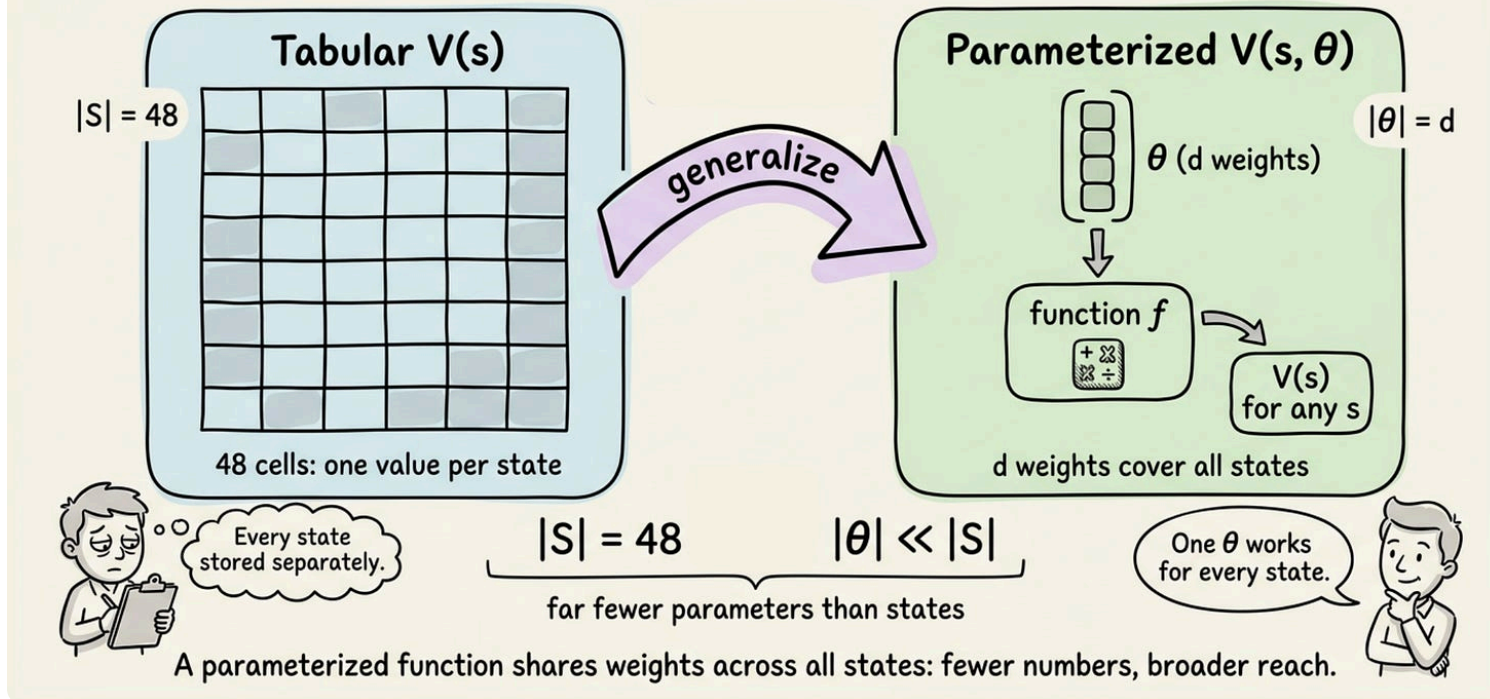
Introduction to Deep RL and DQN

RL Part 6: From linear features to neural networks, and the engineering choices that makes deep value-based RL possible.

Recap

In the previous chapter, we made the transition from tables to parameterized value functions.

Tabular V to Parameterized V



The reasons were structural. Tables do not scale, and they do not generalize. Mountain car has a state space made of two real numbers, so it is impossible to even index into a table for it. And updating one cell of a Q-table tells us nothing about the cell next to it. The fix was to write the value function as a function over states, with a small parameter vector θ controlling its shape.

$$\hat{v}(s, \theta) \approx v_{\pi}(s)$$

We laid out the prediction objective, mean square value error. We then worked through linear function approximation in detail, where the value estimate is the inner product of fixed features and learnable weights.

$$\hat{v}(s, \theta) = \theta^{\top} \phi(s) = \sum_{i=1}^d \theta_i \phi_i(s)$$

From that foundation, we built two learning algorithms:

- Gradient Monte Carlo uses the full return as a target, which makes it a true gradient method on a well-defined squared-error objective.

$$\theta \leftarrow \theta + \alpha [G_t - \hat{v}(S_t, \theta)] \nabla_{\theta} \hat{v}(S_t, \theta)$$

error (points to $G_t - \hat{v}(S_t, \theta)$)
direction (points to $\nabla_{\theta} \hat{v}(S_t, \theta)$)

- Semi-gradient TD(0) replaces the return with a bootstrapped target, gaining online updates and low variance, but introducing the bias of differentiating only the prediction and not the target.

$$\theta \leftarrow \theta + \alpha [R_{t+1} + \gamma \hat{v}(S_{t+1}, \theta) - \hat{v}(S_t, \theta)] \nabla_{\theta} \hat{v}(S_t, \theta)$$

↑
 u_t (points to $R_{t+1} + \gamma \hat{v}(S_{t+1}, \theta) - \hat{v}(S_t, \theta)$)

We then extended our understanding to control:

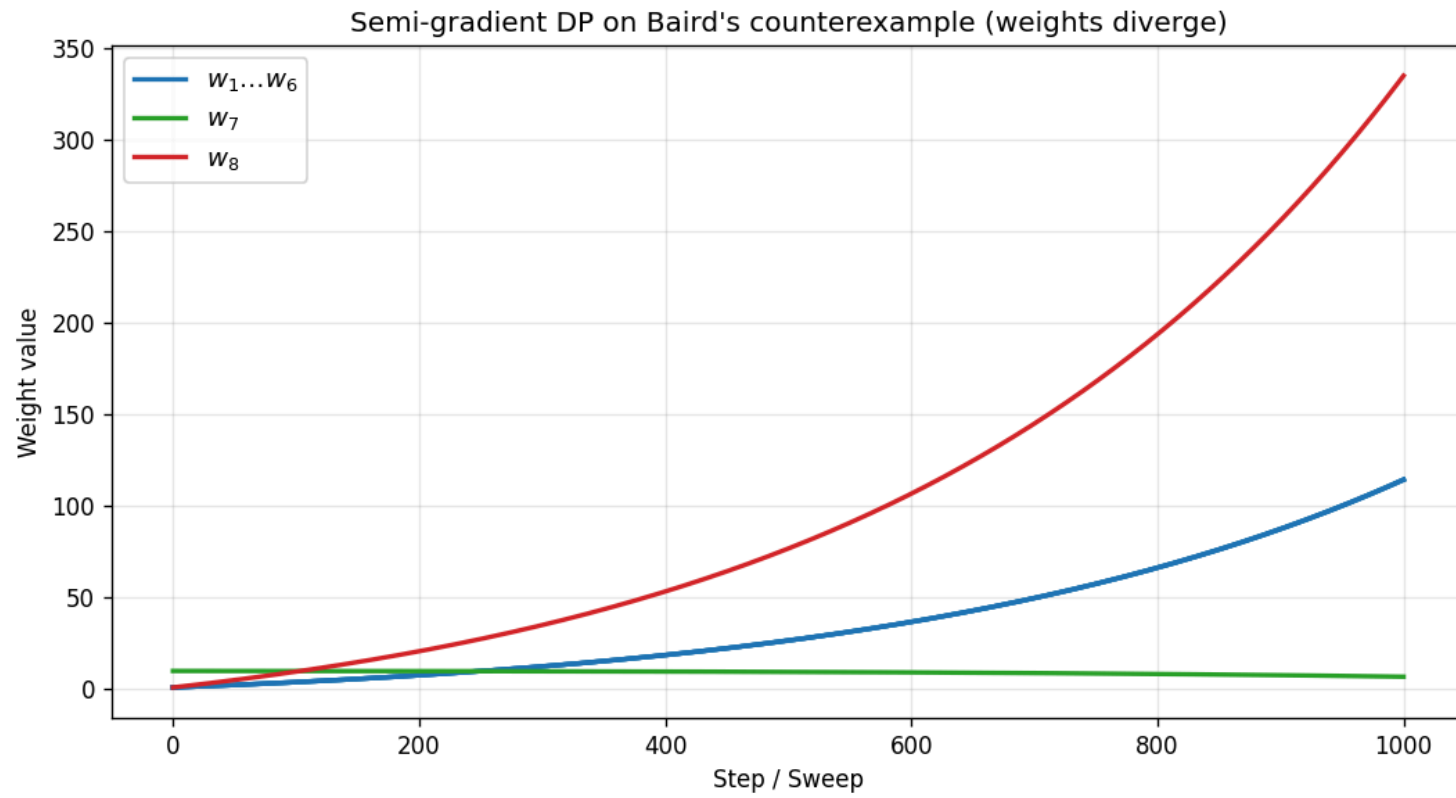
- Semi-gradient SARSA learns the action-value function on-policy and works reliably with linear features.

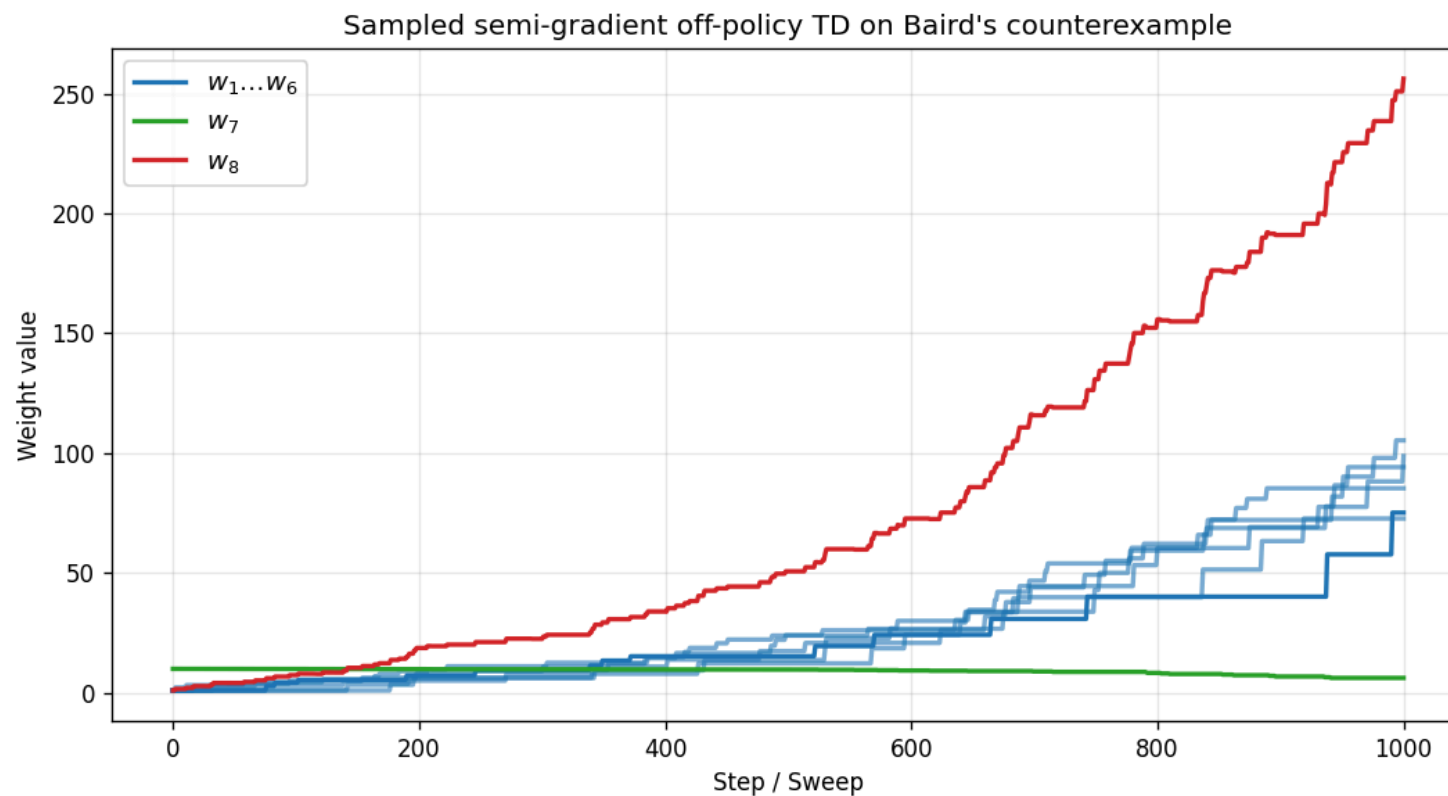
$$\theta \leftarrow \theta + \alpha [R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \theta) - \hat{q}(S_t, A_t, \theta)] \nabla_{\theta} \hat{q}(S_t, A_t, \theta)$$

- Semi-gradient Q-learning uses the max over next actions, making it off-policy. This places it squarely inside what Sutton and Barto call the deadly triad: function approximation, bootstrapping, and off-policy learning combined.

$$\theta \leftarrow \theta + \alpha [R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \theta) - \hat{q}(S_t, A_t, \theta)] \nabla_{\theta} \hat{q}(S_t, A_t, \theta)$$

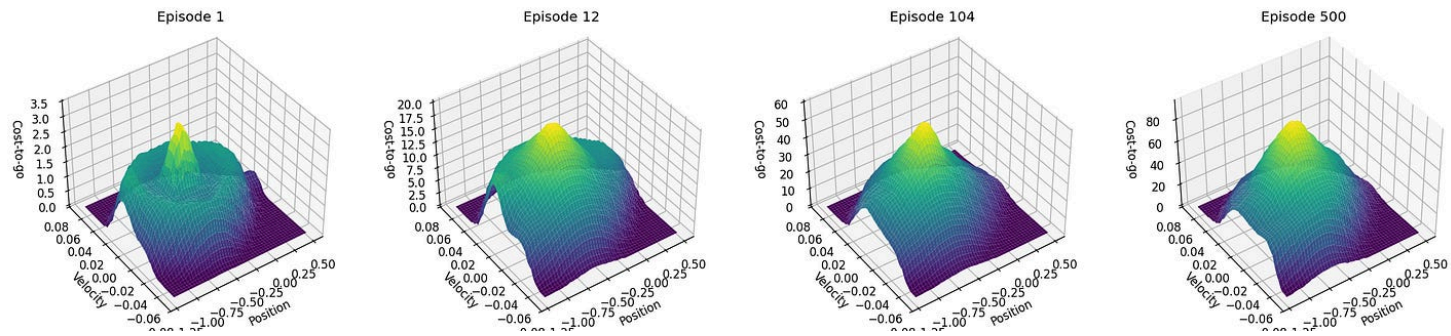
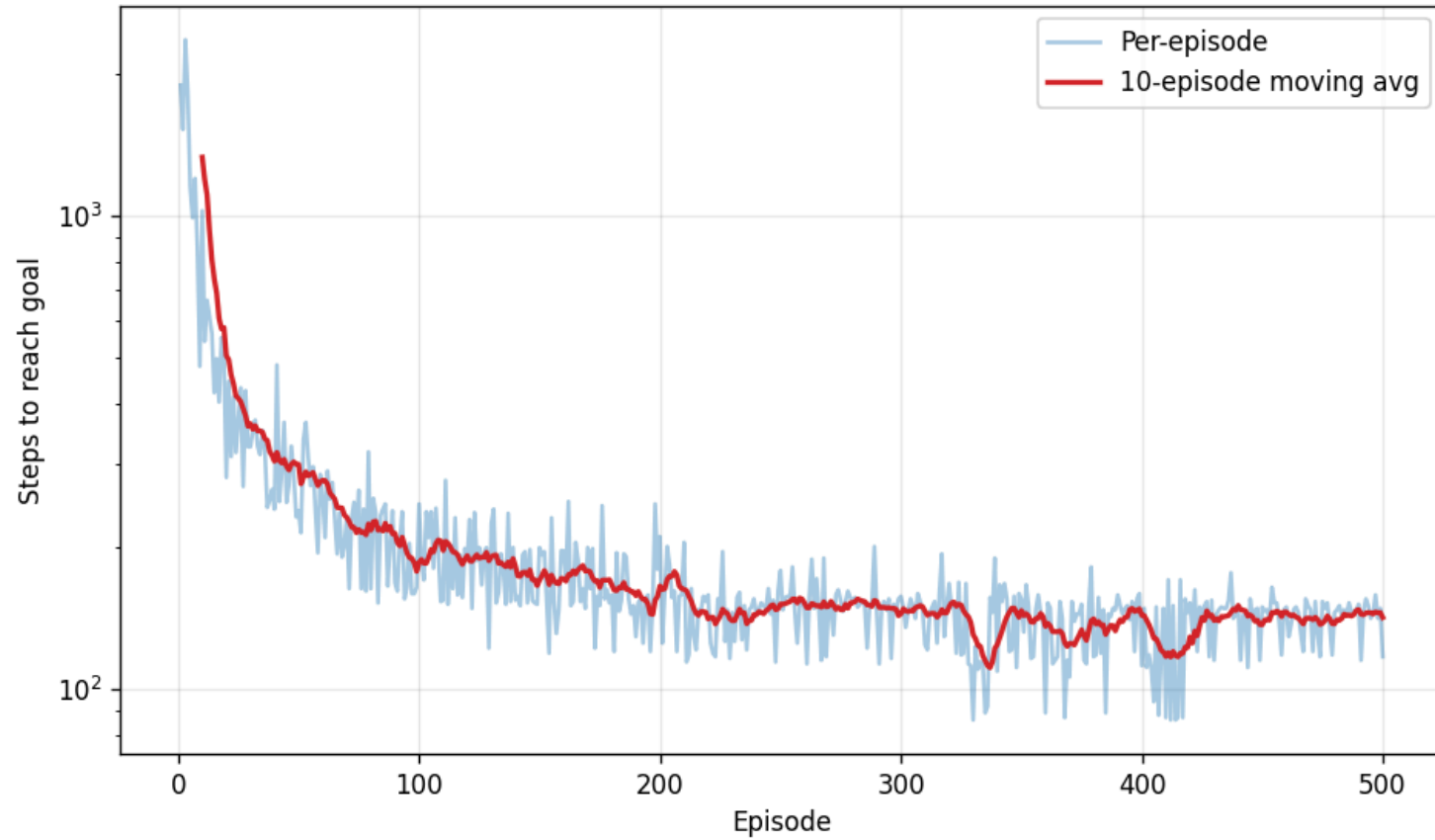
Finally in the hands-on section, we saw the deadly triad cause divergence on Baird's counterexample, a tiny seven-state MDP where the weights grow without bound even though the true value function is zero everywhere.





We also saw the other side of the picture with semi-gradient SARSA on Mountain Car. Tile coding gave us a useful continuous-state representation, and the cost-to-go surface reconstructed the underlying physics of the task.

Mountain Car: semi-gradient SARSA with tile coding

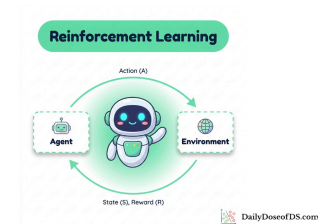


If you have not read Chapter 5, we recommend doing so first:

Function Approximation

RL Part 5: From tables to parameterized value functions.

 Daily Dose of Data Science • Avi Chawla

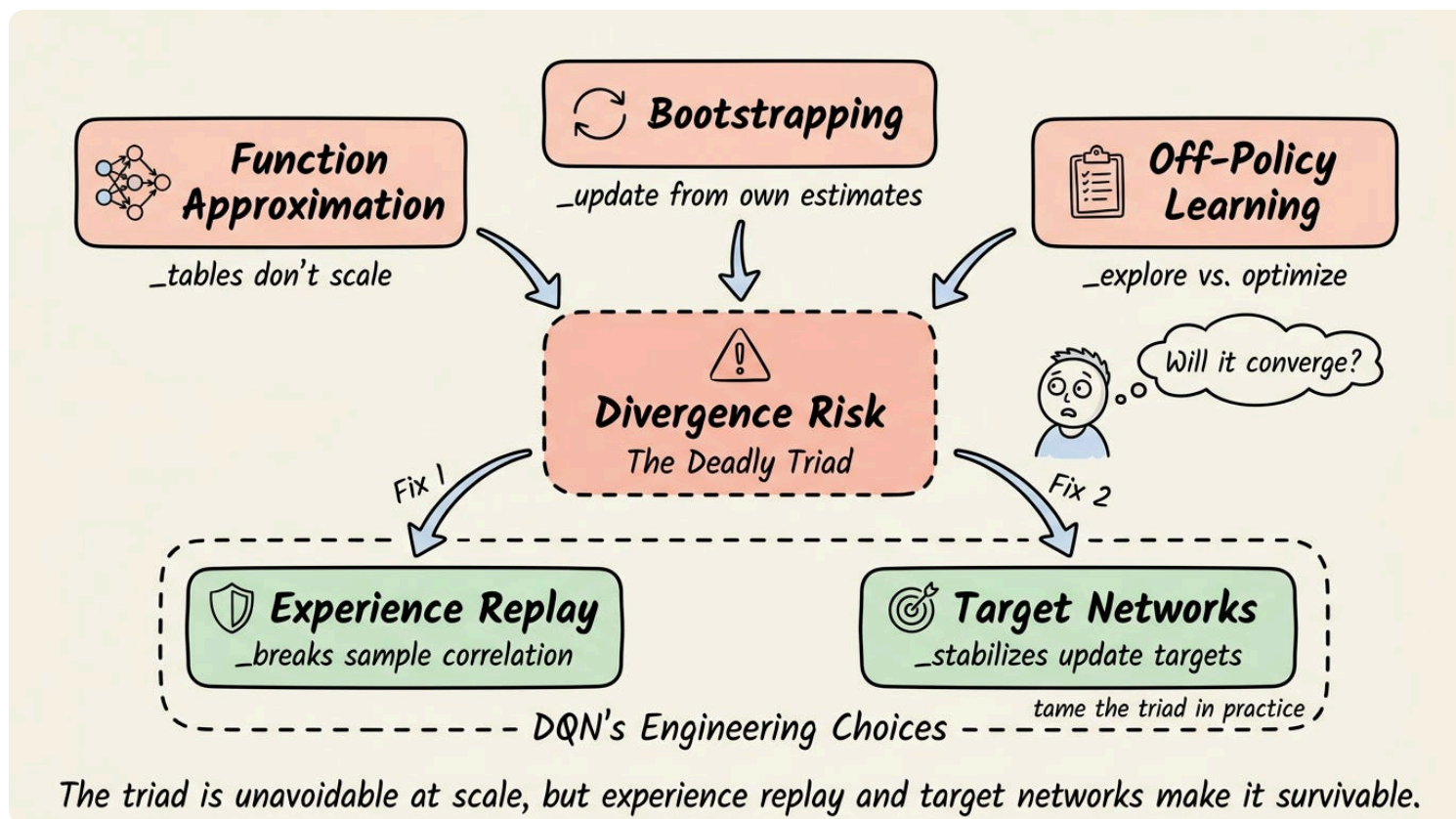


Introduction

The deadly triad demonstration at the end of Chapter 5 made one thing clear. Combining function approximation, bootstrapping, and off-policy learning creates a real risk of divergence, and that combination is exactly what we want for scalable value-based RL:

- We want function approximation because tables do not scale.
- We want bootstrapping because waiting for full returns is slow.
- We want off-policy learning because we want to learn about an optimal greedy policy while exploring with a softer one.

This chapter is about how the field made that combination work in practice.



We will extend the function class from linear to neural, watch new instabilities emerge in the deep setting, and see how DQN's two engineering choices, experience replay and target networks, tame them. The chapter then closes with a hands-on experiment training a DQN agent on CartPole.



CartPole is a classic control theory and reinforcement learning benchmark. The objective is to balance a pole vertically on a moving cart by applying left or right forces.

Let's begin!

From linear to neural

The mechanical step from linear function approximation to neural function approximation is small. We already know the action-value function as a linear combination of features:

$$\hat{q}(s, a, \theta) = \theta^\top \phi(s, a)$$

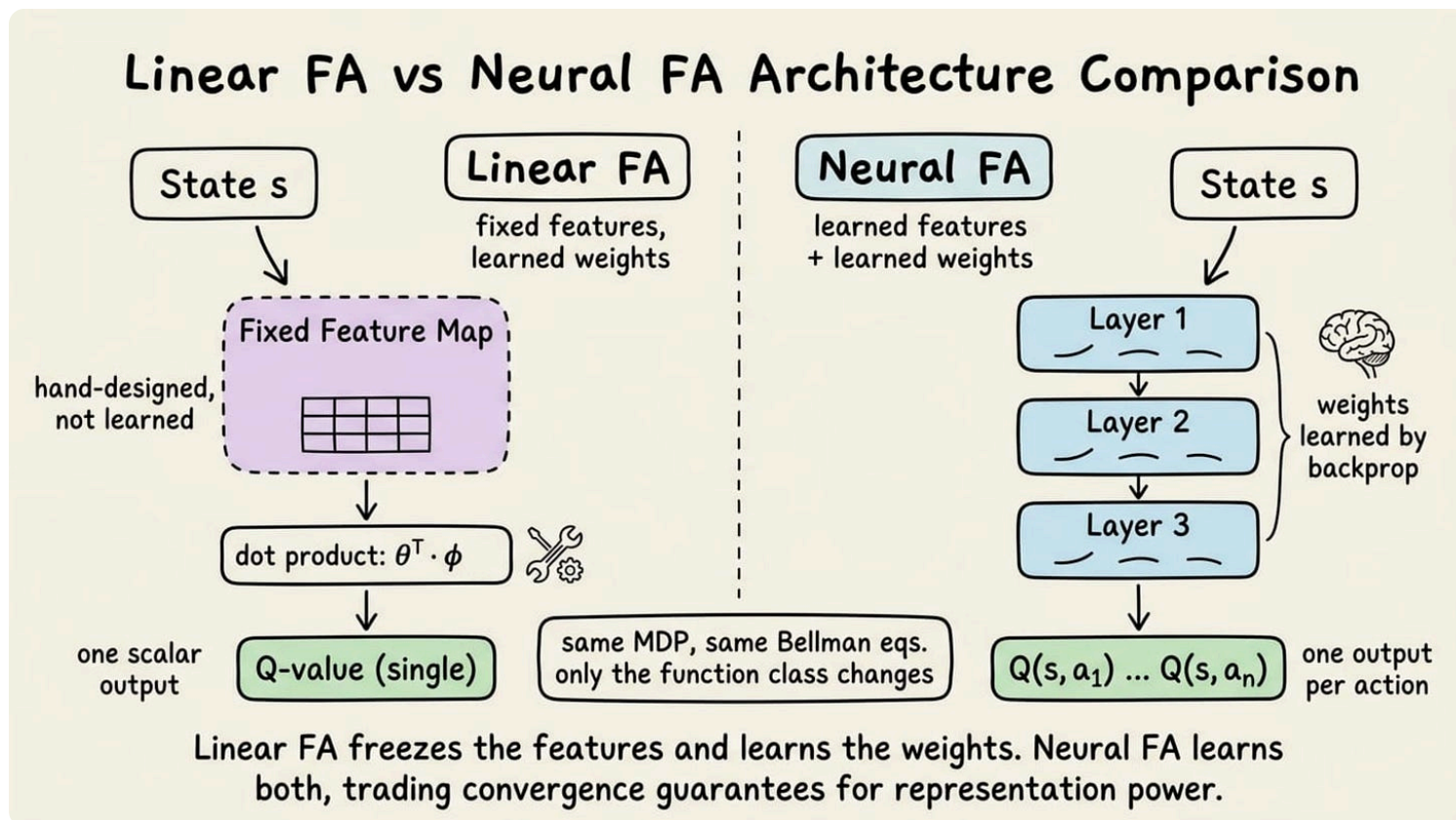
The shift is to replace this expression with a more general parameterized function:

$$\hat{q}(s, a, \theta) = f_{\theta}(s, a)$$

where, f_{θ} is now any function differentiable in θ , typically a neural network.

The structure tells us the key change. With linear FA, the features carried the inductive bias and the weights did the learning. With neural FA, the network does both. Nothing about the underlying RL problem changes.

The MDP is the same, the Bellman equations are the same, only the function class used to approximate the value function has grown.



The reason we want this change is representation learning. Hand-designed features work well when the state space is low-dimensional and we know what matters. But for higher dimensions, we almost always have no idea what the right features are. A neural network discovers them from data.



The gradient computation also changes. With linear FA, the gradient of $\hat{q}$ with respect to θ was just $\phi(s, a)$. With neural FA, the gradient is

computed by backpropagation through the network.

The trade-off is theoretical. In Chapter 5, we noted that linear on-policy semi-gradient TD converges to a unique fixed point. That result depends on linearity. With nonlinear function approximation, even on-policy semi-gradient TD can diverge.

So moving to neural networks gives up the theoretical guarantees we had with linear FA, even before we add off-policy learning back into the mix.

The field accepts this trade-off because the empirical results justify it, and because engineering choices make the situation tractable in practice.

In summary, the move to neural function approximation is a small mechanical change that buys us representation learning, at the cost of the convergence guarantees we had with linear features.

The rest of the chapter is about what goes wrong when we make this move, and what we do about it.

The naive approach and what breaks

Let's try the most direct approach. Take semi-gradient Q-learning from Chapter 5, swap the linear function for a neural network, and run it online.

The update rule is:

$$\theta \leftarrow \theta + \alpha \left[R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \theta) - \hat{q}(S_t, A_t, \theta) \right] \nabla_{\theta} \hat{q}(S_t, A_t, \theta)$$

error 

The terms:

- The bracketed expression is the TD error.
- $\nabla_{\theta} \hat{q}(S_t, A_t, \theta)$ is the gradient of the predicted Q-value with respect to all network parameters, computed by backpropagation.
- α is the step size.

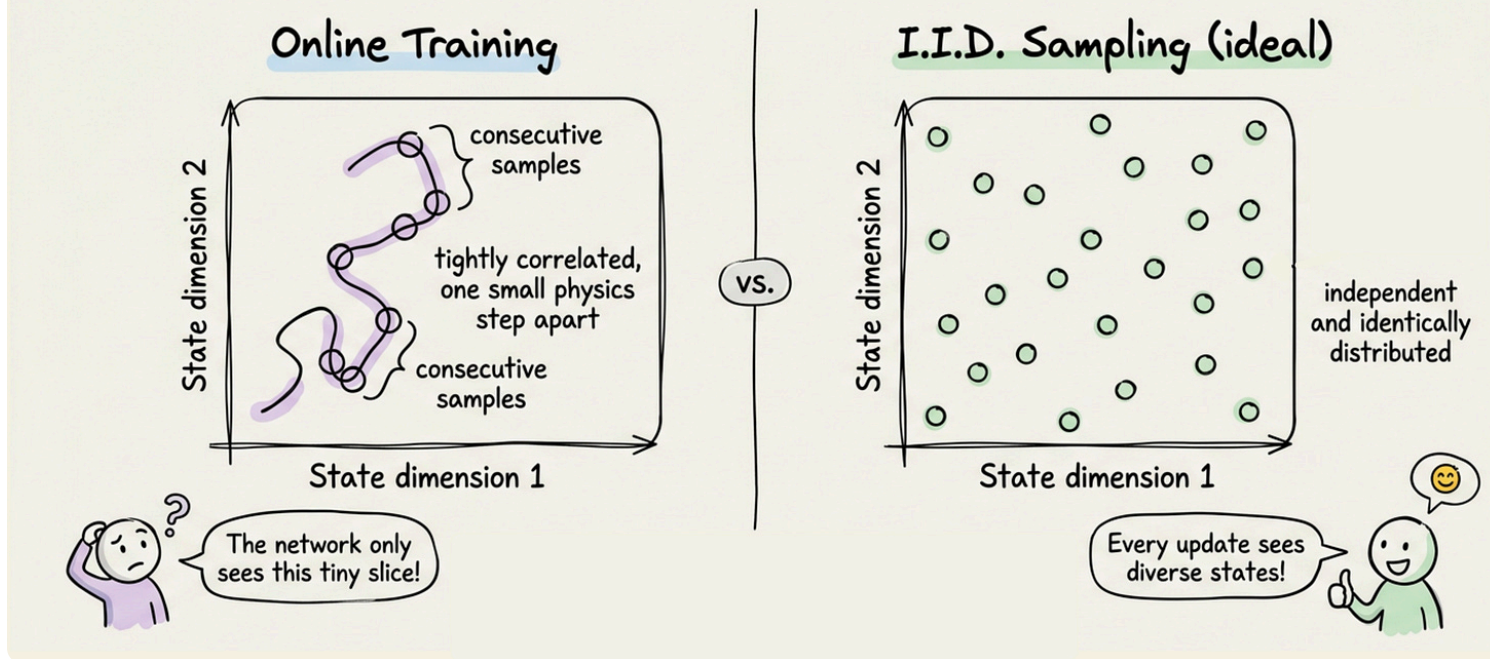
In structure, this is identical to what we had with linear FA. The only difference is what $\hat{q}$ and its gradient look like under the hood.

Run this loop on a problem like CartPole, and the agent learns nothing. Often, the weights drift, the loss climbs, and the agent's behavior gets worse over time.

There are three reasons, and each one maps back to something we already discussed in Chapter 5:

- The first problem is sample correlation. In online learning, the data we train on comes from consecutive transitions in the environment. Two consecutive states in CartPole differ by one small physics step. Thus two consecutive samples are not independent, they are tightly correlated. Stochastic gradient descent assumes (or at least works much better with) approximately independent and identically distributed samples. When samples come in a correlated sequence, the network overfits to whatever local region of the state space the agent happens to be in, and forgets about regions it visited earlier.

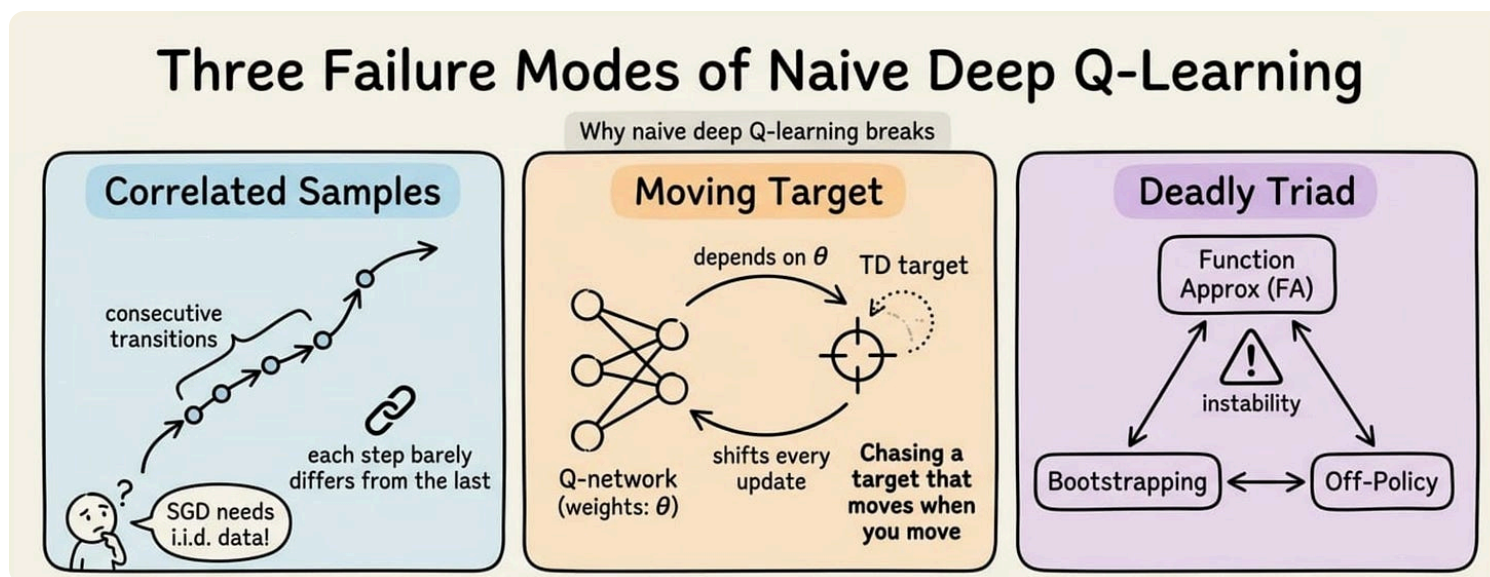
Correlated vs. I.I.D. Sampling in State Space



- The second problem is non-stationary targets. The TD target $R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \theta)$ depends on θ . Every time we take a gradient step, the target shifts too. We are trying to fit a moving target, and the moving target depends on us. In Chapter 5 we spelled this out as the core of the semi-gradient idea: we differentiate only the prediction, not the target. With linear FA and on-policy sampling, this was tolerable. But with neural FA and off-policy sampling, the same trick stops being tolerable. A small change in θ propagates through the network and can change the predicted

value at many states at once, including the next state we are about to bootstrap from. The target now moves in unpredictable directions every update.

- The third problem is the deadly triad in full nonlinear form. We are using function approximation (the network), bootstrapping (the TD target), and off-policy learning (the max over next actions). This is exactly the combination Baird's counterexample showed was unstable, and now we are scaling it up to a much larger function class.



This was the situation in 2013, when Mnih et al. published "Playing Atari with Deep Reinforcement Learning". It was the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning.

The contribution was not the basic idea of combining Q-learning with a neural network. That had been tried before. The contribution was the engineering choices that made it work: experience replay and, in the 2015 Nature version, target networks. We turn to those next.

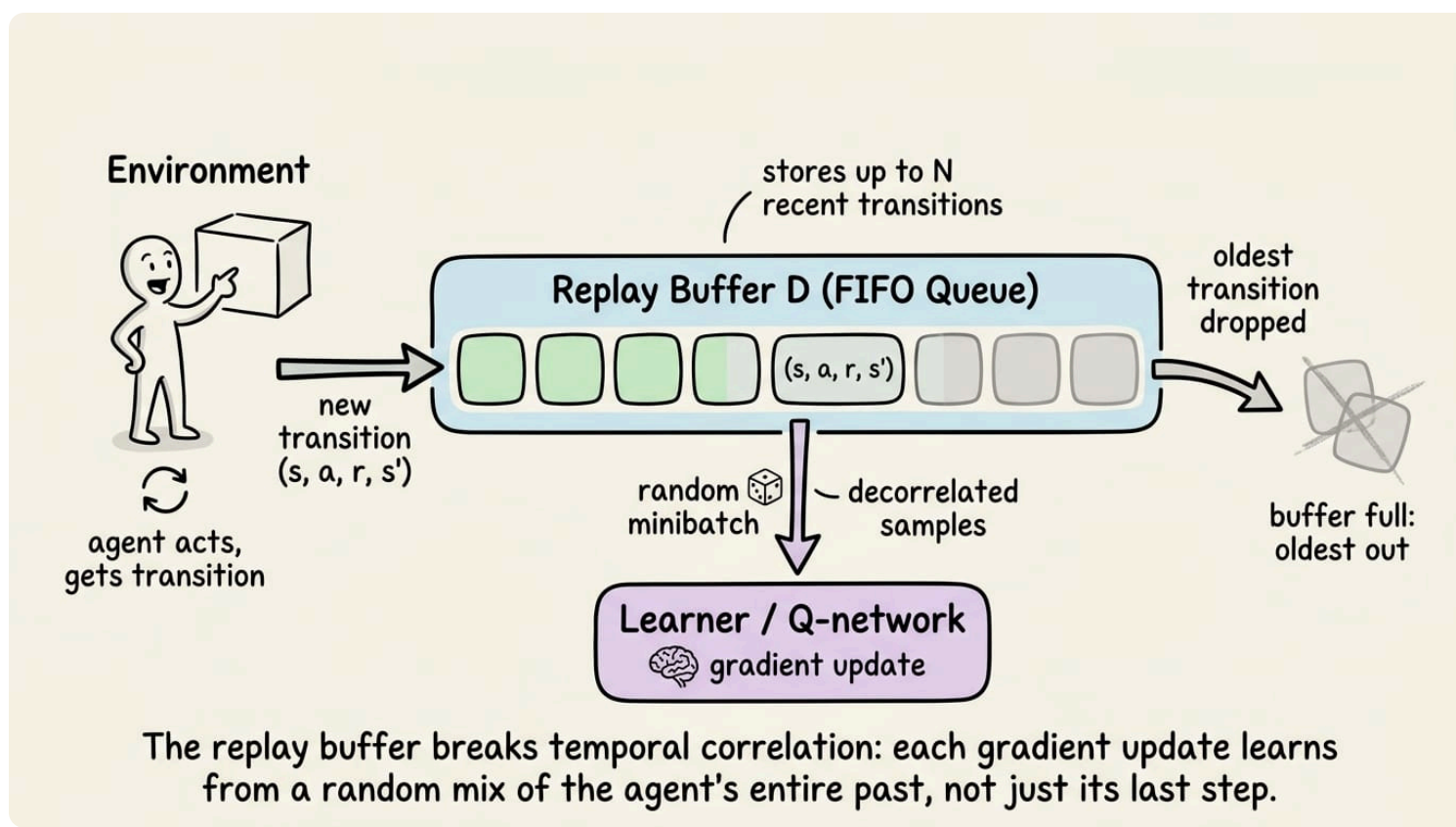
Experience replay

The first of DQN's two engineering choices is experience replay.

The idea is simple. Instead of learning from each transition immediately and then discarding it, the agent stores transitions in a buffer and trains on minibatches sampled randomly from the buffer.

Concretely, the agent maintains a fixed-size buffer $\mathcal{D}$ (a FIFO queue), and at every step it does two things:

- It adds the latest transition to the buffer.
- It samples a minibatch of transitions uniformly at random from the buffer and applies the Q-learning update on that minibatch.



This directly addresses the first problem from the previous section. Transitions in a minibatch sampled randomly from the buffer are no longer consecutive in time.

They come from many different points in the agent's past, possibly from different episodes, possibly from different regions of the state space. The samples in a single gradient update are approximately decorrelated, which is much closer to what stochastic gradient descent expects.

Experience replay also gives us sample efficiency. Without replay, each transition is used exactly once before being discarded. With a buffer of N transitions, each transition gets used multiple times across many gradient updates. The same data drives more learning.



There is something subtle going on here. Experience replay only works because Q-learning is off-policy. When we sample a transition from the buffer, we are sampling from the distribution of states and actions that some past version of the policy produced. That distribution differs from what the current policy would produce. An on-policy algorithm like SARSA cannot tolerate this mismatch cleanly, because its update implicitly assumes the action A_{t+1} at the next state was drawn from the current

policy. Q-learning's max operator does not care which action the behavior policy actually took.

The trade-off, however, is that experience replay pushes Q-learning further into the deadly triad.

The samples in the buffer were generated by a behavior policy that differs from the current target policy, and that mismatch grows the longer a transition sits in the buffer.

So while replay solves the correlation problem cleanly, it does not solve the deadly triad on its own. For that, we need the second engineering choice.

Target networks

The idea here is to maintain two copies of the Q-network:

- The online network Q_θ has its parameters updated at every step.

- The target network Q_{θ^-} has its parameters held fixed, and only periodically copied from θ .

In the TD target, we use the target network instead of the online network:

$$R_{t+1} + \gamma \max_{a'} Q_{\theta^-}(S_{t+1}, a')$$

where, θ^- is the target network's parameters, updated less frequently.

The loss minimized at each gradient step is then:

expectation is taken over
transitions sampled from the replay buffer

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_{\theta}(s, a) \right)^2 \right]$$

The expectation is taken over transitions sampled from the replay buffer $\mathcal{D}$. The gradient is taken with respect to θ only, because θ^- is treated as a constant. This is the same semi-gradient idea from Chapter 5. The target does not depend on the current θ . It depends on θ^- .

This directly addresses the second problem from earlier. The target is no longer a moving function of the parameters we are updating. Between target updates, it is a fixed function.

We are doing supervised regression toward a fixed target for C steps, then refreshing the target, then doing supervised regression toward a slightly different fixed target for the next C steps. The training signal looks much more like ordinary deep learning, which is something the optimizer can handle.

There are two common ways to refresh the target network:

- Hard updates copy θ into θ^- every C steps, then leave θ^- frozen until the next copy.
- Soft updates do a slow exponential average at every step:
$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$$
 for some small τ (like 0.005).



Hard updates are simpler and were used in the original DQN paper. Soft updates are smoother and avoid the discontinuous jumps in the target, at the cost of introducing τ as another hyperparameter.

The trade-off is that the target network slows down learning.

In summary, the target network combined with experience replay gives us the two pieces we need to make deep Q-learning possible.

The DQN algorithm

Now let us put the pieces together. Deep Q-Network, is the combination of Q-learning, a deep neural network for the action-value function, experience replay, and a target network. Here is the algorithm:

1. Initialize the online network Q_θ with random weights.
2. Initialize the target network Q_{θ^-} as a copy of Q_θ .
3. Initialize the replay buffer $\mathcal{D}$ as an empty FIFO queue with capacity N .
4. For each environment step t :
 - Select an action A_t using ϵ -greedy policy on $Q_\theta(S_t, \cdot)$.
 - Execute A_t in the environment, observe reward R_{t+1} and next state S_{t+1} .
 - Store the transition $(S_t, A_t, R_{t+1}, S_{t+1}, \text{done})$ in $\mathcal{D}$.
 - Sample a minibatch of transitions (s, a, r, s', d) from $\mathcal{D}$.
 - For each transition in the minibatch, compute the target $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$ if not terminal, else $y = r$.
 - Take a gradient step on the loss $\mathcal{L}(\theta) = \frac{1}{|\text{batch}|} \sum (y - Q_\theta(s, a))^2$ (or Huber loss in practice).
 - Every C steps, copy θ into θ^- .

A few details deserve unpacking:

- The terminal-state handling matters. When a transition ends an episode (the `done` flag is true), there is no next state to bootstrap from. The target is just the observed reward, with no $\gamma \max$ term.



If we forget this, the agent ends up bootstrapping off whatever garbage the target network predicts for impossible states, and learning silently degrades.

- The Huber loss is a robustness choice. It uses a squared term when the error is small (below a threshold) and a linear term when the error is large. This makes it less sensitive to outliers than mean squared error, and in some cases prevents exploding gradients.



Early in training, TD errors can be very large. Squaring them produces enormous gradients that destabilize the network. Huber loss keeps the gradients bounded once the error exceeds the threshold.

- The exploration uses standard ϵ -greedy on the online network. Typically, ϵ starts near 1.0 (almost all random actions) and decays linearly to a small value like 0.05 over the first part of training. The target network is never used for action selection. It only computes the bootstrap target.



The choice of C is one of the most important hyperparameters in DQN. Too small, and the target network behaves like the online network, and we are back to the moving-target problem. Too large, and learning slows to a crawl because the bootstrap signal is stale. Hence, given a problem, it could require a bit of tuning before you actually start seeing what you want.

Limits of DQN

Despite its success, DQN has limits:

- The first limit is continuous action spaces. DQN's bootstrap target uses $\max_{a'} Q_{\theta^-}(s', a')$, which requires enumerating all actions at the next state. With a discrete action space it is fine, but with a continuous action space, that becomes an issue. There are infinitely many actions to consider, and no closed-form solution to picking the best one.
- Another limitation is sample inefficiency. This means vanilla DQNs often require too many (millions of) environment interactions to learn anything.
- Exploration is also a major limitation. ϵ -greedy only finds rewards that random actions can stumble into. Solving several RL problems requires more structured exploration. But that's a substantial topic in its own right.

In summary, DQN is not a finished story. It is a specific algorithm that works well on a specific class of problems: discrete actions, dense rewards, and simulators where you can afford millions of interactions.

We have now understood all the key concepts for this chapter. The pieces are in place. So let's go ahead and dive into hands-on section for some practical experimentation.

Hands-on: DQN on CartPole

For our hands-on demonstration, we will implement DQN and train it on CartPole.

The code and project setup are attached below as a zip file. You can extract it and run `uv sync` to get going.

```
.gitignore  
.python-version  
pyproject.toml  
train.py  
training_curves.png  
uv.lock
```

Download the zip file below:

dqn

dqn.zip • 130 KB

For details about versions and dependencies, check the `.python-version` and `pyproject.toml` files.

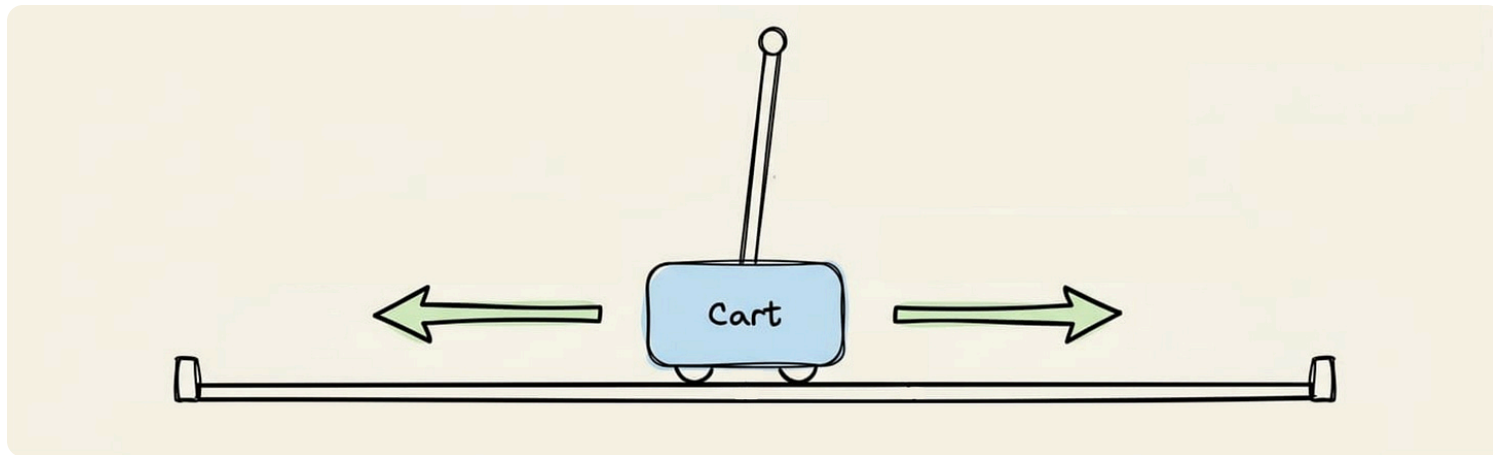
The environment

CartPole-v1 is a classic control benchmark from Gymnasium. A cart moves along a one-dimensional track, with a pole attached to it by a frictionless hinge. The goal is to keep the pole upright by pushing the cart left or right.

The state is four-dimensional:

- cart position
- cart velocity
- pole angle
- pole angular velocity

The action space has two discrete actions: push left (0) and push right (1). The reward is +1 for every step the pole stays up. The episode ends if the cart position exceeds the bounds, the pole angle exceeds ± 0.2095 radians, or the episode reaches 500 steps.



A return of 500 means the pole stayed up for the entire episode (the maximum). The default reward threshold for considering the task solved is 500 for v1.

Code

Let's now take a look at the key portions of our training script. Starting with defining the Q-network:

QNetwork defines the action-value function as a small MLP. Two hidden layers of 128 units each, with ReLU activations.

The output layer has one unit per action (here, two: push left and push right). The forward pass returns Q-values for all actions at the given state.

Next, let's define the replay buffer:

`ReplayBuffer` is a thin wrapper around `collections.deque` with `maxlen` set to the capacity.

The `push` method adds a new transition; old transitions fall off the other end automatically when the buffer fills up.

The `sample` method draws a random minibatch and stacks the components into tensors for the network.

Moving ahead:

Here:

- The hyperparameters here are fine for small environments:
 - Learning rate $5e-4$
 - Discount factor 0.99
 - Replay buffer capacity 10,000 transitions.
 - Batch size 128
 - Epsilon decays linearly from 1.0 to 0.01 over the first 10,000 steps, after which exploration stays at 1%.

- `LEARNING_STARTS` is the number of env steps (1000) we collect before starting gradient updates, so the replay buffer has a sufficient pool of transitions to sample from. Without it, we'd train on a tiny, highly correlated batch.
- `TAU` is the soft update hyperparameter.
- `epsilon_by_step` computes the linear epsilon schedule. The agent explores randomly almost all the time at the start, then increasingly relies on its learned Q-values.
- `select_action` implements ϵ -greedy. With probability ϵ , return a random action. Otherwise, pass the state through the online network and pick the action with the highest predicted Q-value. The `torch.no_grad()` block prevents the network from tracking gradients during action selection, since we're not training here.
- `train_step` is where the DQN update happens. After sampling a minibatch from the buffer, it computes the current Q-values for the actions actually taken (`gather` picks out the right column from each row). Then, under `torch.no_grad()`, it computes the bootstrap target using the target network, with the γ term zeroed out for terminal transitions. The loss is Huber (`smooth_l1_loss` in PyTorch). Gradient clipping with a norm cap of 10

prevents very large updates from destabilizing training. Finally the soft update (Polyak averaging) of target network parameters.

Finally, let's take a look at the `main` function:

`main` ties it all together. It creates the environment, builds the two networks, and runs the episode loop.

At every environment step, it selects an action, stores the transition, and takes a training step (after `LEARNING_STARTS` steps have accumulated, so the buffer has enough data).



One detail worth flagging: when storing the `done` flag in the replay buffer, we use `terminated` (the pole fell or the cart left bounds), not `terminated or truncated`. In Gymnasium, `truncated` means the episode hit the time limit (500 steps), not that the state was actually terminal. If the agent successfully balances for 500 steps, the next state is still a perfectly valid state, just one the agent did not get to observe. Treating it as terminal (zeroing the bootstrap) would incorrectly tell the agent that those states have no future value. This is a subtle bug that could hurt performance if you get it wrong.

Running the experiment

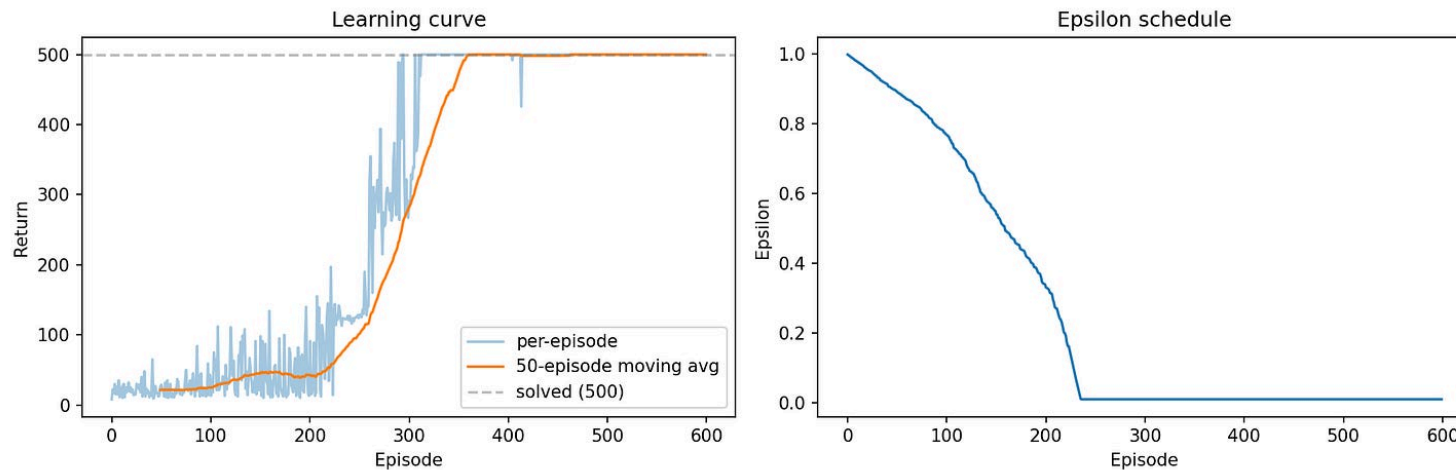
Running `train.py` with the default seed produces the following output on our machine:

Episode	25		avg return (last 25):	20.0		eps: 0.951
Episode	50		avg return (last 25):	22.6		eps: 0.895
Episode	75		avg return (last 25):	20.8		eps: 0.843
Episode	100		avg return (last 25):	29.3		eps: 0.771
Episode	125		avg return (last 25):	43.2		eps: 0.664
Episode	150		avg return (last 25):	45.8		eps: 0.550
Episode	175		avg return (last 25):	43.5		eps: 0.443
Episode	200		avg return (last 25):	41.6		eps: 0.340
Episode	225		avg return (last 25):	77.3		eps: 0.149
Episode	250		avg return (last 25):	124.6		eps: 0.010
Episode	275		avg return (last 25):	229.0		eps: 0.010
Episode	300		avg return (last 25):	329.4		eps: 0.010
Episode	325		avg return (last 25):	439.7		eps: 0.010
Episode	350		avg return (last 25):	500.0		eps: 0.010
Episode	375		avg return (last 25):	500.0		eps: 0.010
Episode	400		avg return (last 25):	500.0		eps: 0.010
Episode	425		avg return (last 25):	496.7		eps: 0.010
Episode	450		avg return (last 25):	500.0		eps: 0.010
Episode	475		avg return (last 25):	500.0		eps: 0.010
Episode	500		avg return (last 25):	500.0		eps: 0.010
Episode	525		avg return (last 25):	500.0		eps: 0.010
Episode	550		avg return (last 25):	500.0		eps: 0.010
Episode	575		avg return (last 25):	500.0		eps: 0.010
Episode	600		avg return (last 25):	500.0		eps: 0.010



The exact values can vary across machines, but the behavior would mostly be consistent.

Also taking a look at the graphs:



- First, the random phase. From episode 1 to roughly episode 200, the agent is exploring with high ϵ , the buffer is filling up, and average returns sit around 20 to 40. The Q-network is still essentially noise.
- Then is the learning phase. Roughly around episode 200 to 400, as ϵ decays further and the buffer accumulates more diverse transitions, the agent's policy improves, thus improving the returns.
- Post that the returns in our run mostly stayed at 500, however the case might be different on your machine.



Since exact values and ranges can vary across machines, what to note here is actually the fact that our DQN learns as episodes pass.

Based on our learning and experimentation so far, we recommend exploring the following topics as self-learning exercises:

- Vanilla DQN and catastrophic forgetting.
- The variants that improved vanilla DQN:
 - Double DQN
 - Dueling DQN

With that, the hands-on section concludes, and so does this chapter. In the upcoming chapters, we will continue to build on the core ideas of RL and explore concepts and their implementations wherever applicable.

Conclusion

In this chapter, we explored the move from linear value function approximation to neural function approximation, and the engineering choices that made deep value-based RL possible in practice.

We started by noting that the mechanical step from linear to neural is small. The form of the value function generalizes from $\theta^\top \phi(s)$ to $f_\theta(s)$, and the gradient is computed by backpropagation. What we lose in this move is the theoretical convergence guarantees that held for linear on-policy semi-gradient TD.

We then walked through what breaks when we naively combine deep Q-learning with online updates: sample correlation, non-stationary targets, and the deadly triad scaled up.

The two engineering choices that fix this are experience replay and target networks. Experience replay stores transitions in a buffer and samples minibatches uniformly at random, decorrelating the training data and improving sample efficiency. Target networks maintain a frozen copy of the Q-network, used only to compute the bootstrap target, making Chapter 5's semi-gradient pretense literal.

The full DQN algorithm puts these pieces together: Q-learning with a neural network for the action-value function, ϵ -greedy exploration, experience replay, a

target network updated every C steps, and the Huber loss for robustness.

Finally, the hands-on section implemented DQN from scratch in PyTorch and trained it on CartPole.

Continuing our journey further, in the next chapter, we will explore and learn about policy gradients.

The aim, as always, is to build a strong conceptual foundation and equip you with a flexible framework for reasoning about the core principles and the broader landscape in general.

As always, thanks for reading!

Any questions?

Feel free to post them in the comments.

Or

If you wish to connect privately, feel free to initiate a chat here:



A daily colour
practices on
professionals



A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students.

chat icon



Published on May 31, 2026

 Comments

 Share

Menu

Contact

FAQ

Previous

< **Function Approximation**

Next

Policy Gradients: REINFORCE and Actor-Critic >



Daily Dose of Data Science © 2026